Seguridad en Sistemas, Aplicaciones y el Big Data

Tema 7. Análisis de seguridad de las aplicaciones web

Índice

Esquema

Ideas clave

7.1. Introducción y objetivos

7.2. Análisis de seguridad de las aplicaciones web

7.3. Software composite analysis (SCA)

7.4. Análisis de seguridad estático (SAST)

7.5. Análisis funcional de requisitos de seguridad

7.6. Análisis de seguridad dinámico (DAST)

7.7. Análisis de seguridad interactivo (IAST)

7.8. Herramientas de análisis de la seguridad híbridas (HAST)

7.9. Gestión y correlación de vulnerabilidades

7.10. Referencias bibliográficas

A fondo

Análisis de seguridad estático en profundidad

OWASP ZAP vídeos

Herramientas de análisis de seguridad IAST

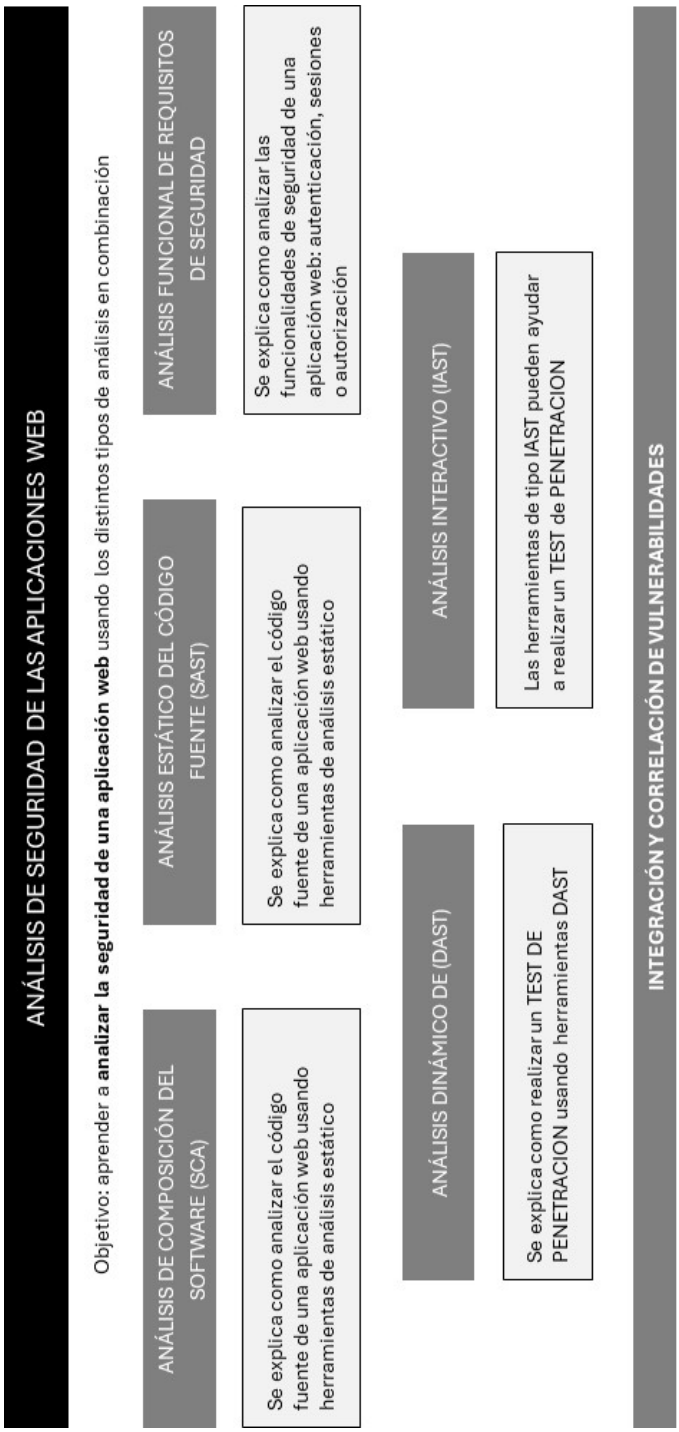
Comparativa de eficacia de herramientas SAST, DAST e IAST

Aplicaciones web para aprendizaje. Juice Shop

Web Application Security Testing Guide

Test

Esquema



© Universidad Internacional de La Rioja (UNIR)

7.1. Introducción y objetivos

Un enfoque de análisis manual de seguridad, como el que se propone en el libro de sería un intento de ***hackeo* ético** de la aplicación desplegada. Un análisis de seguridad puramente manual necesita de personal muy especializado y es muy costoso en tiempo para cubrir toda la superficie de ataque de la aplicación. Por tal motivo, es necesario adoptar un proceso de ciclo de vida de desarrollo seguro (SSDLC) para llevar a cabo otros tipos de análisis de caja blanca para cubrir la mayor parte de la superficie de ataque (Chess, 2007).

El caso más deseable para atajar las vulnerabilidades de una aplicación web es la **implementación de medidas de prevención,** por parte de los desarrolladores, en la fase de desarrollo del código fuente de la aplicación. Es fundamental una formación adecuada en seguridad en el código fuente y de las configuraciones para evitar cometer errores de programación y que supongan vulnerabilidades.

A pesar de las medidas de prevención aplicadas, siempre quedarán vulnerabilidades en el código fuente y en las configuraciones de la aplicación. Por tanto, no hay más remedio que, una vez desarrollada la primera versión de la aplicación o partes de esta, pasar a la detección de vulnerabilidades usando diversos tipos de análisis de seguridad en combinación que se apoyen en herramientas semiautomáticas especializadas de revisión de código fuente o ejecutable de tipo estático, de tipo dinámico en tiempo de ejecución y otras que se verán más adelante.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Después de las tareas de detección usando los distintos métodos y herramientas de análisis de seguridad disponibles, las vulnerabilidades descubiertas han de parchearse, lo que también supone un costo económico y de tiempo. Sin embargo, siempre será más barato hacerlo en el tiempo de desarrollo de la aplicación que una vez desplegada en producción a *posteriori.*

En la Figura 1 se muestra el incremento de costo de arreglo de vulnerabilidades en distintas **fases de la vida del *software.*** Se aprecia que se multiplica por sesenta el costo en el período de la aplicación ya en servicio en relación con el tiempo de diseño de esta.

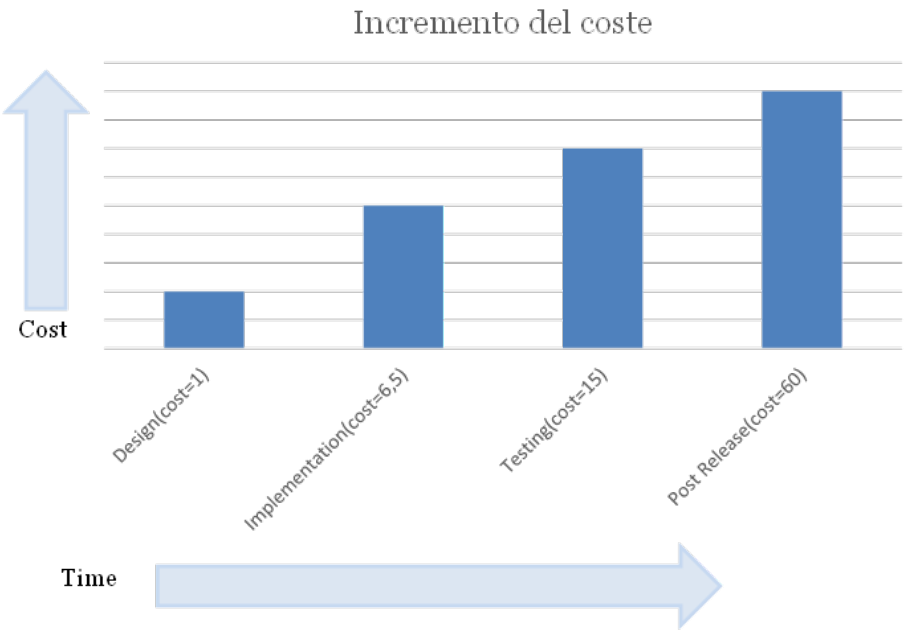


Figura 1. Incremento del coste. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Los principales objetivos de este tema son los siguientes:

- Aprender a realizar un análisis de la seguridad de una aplicación web usando distintos tipos de análisis.
- Conocer los tipos de análisis de seguridad.
- Aprender cómo usar las herramientas de análisis de tipo estático.
- Aprender cómo usar las herramientas de análisis de comprobación de dependencias.
- Aprender cómo usar las herramientas de análisis de tipo dinámico.
- Aprender cómo usar las herramientas de análisis de tipo interactivas.

© Universidad Internacional de La Rioja (UNIR)

7.2. Análisis de seguridad de las aplicaciones web

A la hora de realizar un análisis de seguridad de una aplicación, cubrir toda la superficie de ataque de una aplicación es difícil, de tal forma que es prácticamente imposible detectar el 100 % de vulnerabilidades de seguridad. Un análisis de seguridad de una aplicación web debe realizarse lo más **automáticamente** posible utilizando distintos tipos de herramientas, como SAST, DAST, IAST e híbridas (HAST). Además, se deben incluir comprobaciones manuales para la auditoría de los **resultados obtenidos** con cada tipo, para el descarte de **falsos positivos** y también comprobaciones manuales específicas de la **funcionalidad de seguridad** con ayuda de utilidades incorporadas en las herramientas DAST, de fuerza bruta basadas en diccionario, *fuzzers,* inyectores de código SQL, decodificadores, etc. Para todas estas acciones, se dispone de la ayuda que proporciona la *Guía de prueba de seguridad OWASP.* La idea principal que se debe tener en cuenta para realizar con garantías un análisis de seguridad de una aplicación web, es adoptar un **ciclo de vida de desarrollo seguro** (SSDLC), el cual obliga a llevar a cabo un análisis de seguridad híbrido combinado**.** En la Figura 1 se muestran los tipos de análisis y herramientas de seguridad relacionadas que emplear.

© Universidad Internacional de La Rioja (UNIR)

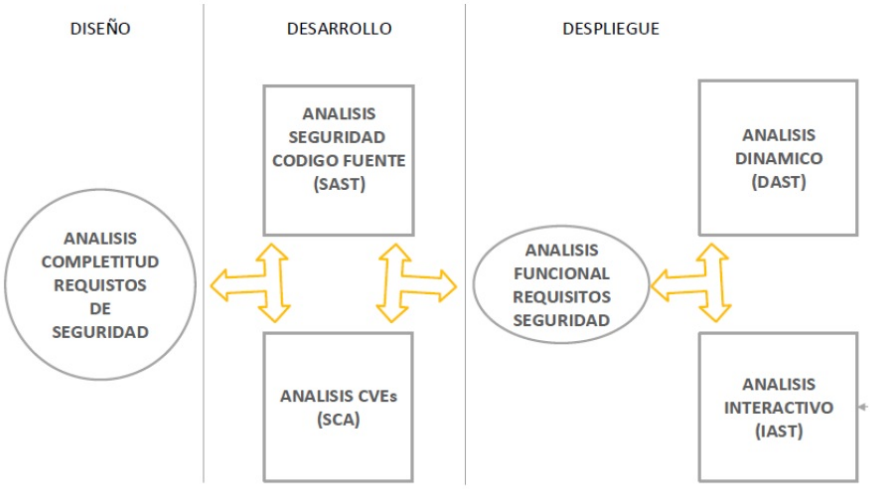


Figura 1. Análisis de seguridad de una aplicación web y ciclo de vida de desarrollo seguro (SSDLC).

Fuente: elaboración propia.

Para acometer el análisis de seguridad en el marco del SSDLC, se deben llevar a cabo los siguientes **tipos de análisis de seguridad:**

- **Fase de análisis diseño:** análisis de completitud de requisitos de seguridad.
- **Fase de desarrollo:**
  - Se revisa la seguridad del código fuente con herramientas de **análisis estático** de caja blanca (SAST).
  - Se comprueban las **vulnerabilidades** CVE de las dependencias de la aplicación Software Composite Analysis (SCA).

© Universidad Internacional de La Rioja (UNIR)

- **Fase de pruebas:**
  - Se lleva a cabo el **análisis funcional de los requisitos de seguridad** que prueba cada requisito y dimensión de seguridad de la aplicación (autenticación, sesiones, autorización, registro, configuraciones de seguridad, etc.).
  - Por último, se realiza la **prueba de penetración** con herramientas de análisis dinámico (DAST) y de caja negra en conjunto con herramientas de análisis interactivo (IAST) de caja blanca.
- **Correlación de los resultados** de detección de vulnerabilidades de todos los tipos de análisis.

La Figura 2 refleja el **resultado** de llevar a cabo un análisis de seguridad según el método mostrado en la figura 1.

© Universidad Internacional de La Rioja (UNIR)

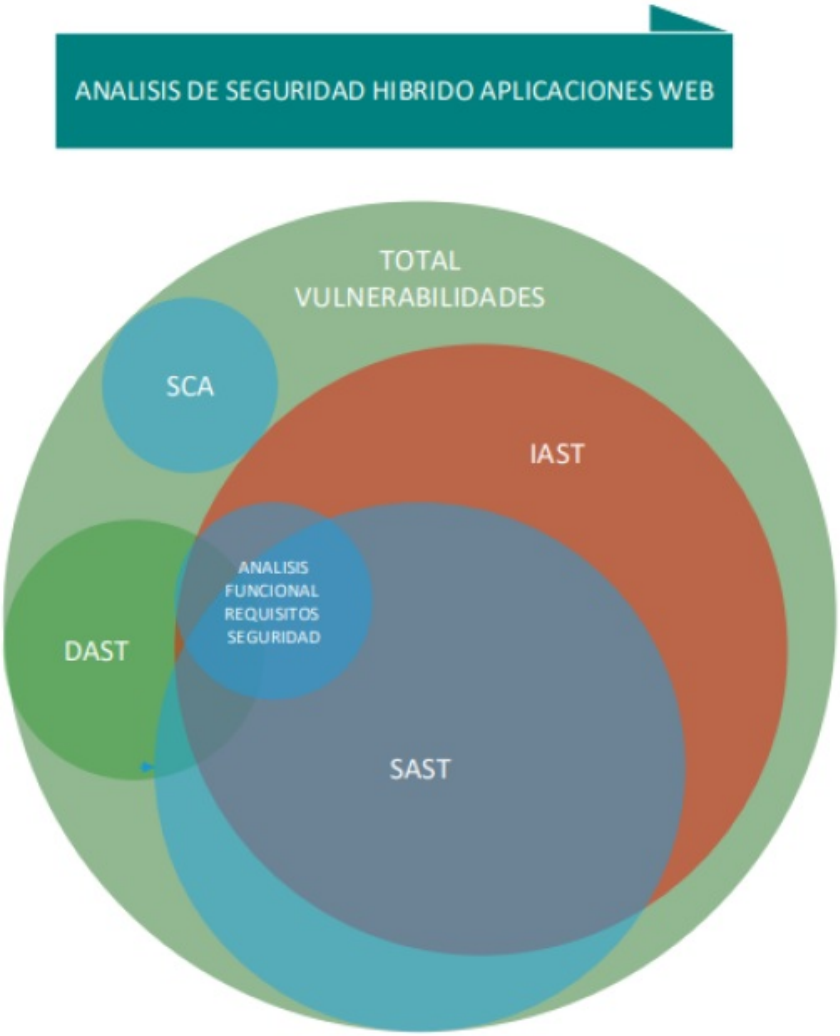


Figura 2. ¿Es posible detectar el 100 % de vulnerabilidades? Fuente: elaboración propia.

A continuación, se enumeran y describen de forma breve, los **tipos de herramientas** de análisis de seguridad por emplear en un **análisis combinado** de seguridad:

- **SAST***(static application security testing):* análisis estático, caja blanca, estudia el código fuente, binarios o *bytecode* sin ejecutar la aplicación. Permite encontrar defectos temprano (inyecciones, manejo inseguro de datos, etc.).

© Universidad Internacional de La Rioja (UNIR)

- **DAST***(dynamic application security testing):* pruebas durante la ejecución, caja negra, se ataca la aplicación en vivo (o en entorno de pruebas) simulando ataques externos.

- **IAST***(interactive application security testing):* una mezcla—sensores/*agents* que monitorizan la aplicación en tiempo de ejecución mientras se realizan pruebas (o uso normal) que combinan visibilidad del código con análisis dinámico.

- **SCA***(software composition analysis):* análisis de componentes de terceros—librerías, dependencias, paquetes, licencias; detecta vulnerabilidades conocidas (CVE), versiones desactualizadas, problemas de licencias, etc.

- **Correlación de vulnerabilidades:** herramientas que son capaces de integrar vulnerabilidades de distintos tipos de análisis para obtener un informe de análisis de conjunto.

En la Figura 3 se resumen las **principales características** de cada tipo de herramienta:

Metodología	Tipo de Prueba	¿Qué Analiza?	Momento de Aplicación	Visibilidad
SAST	Estática	Código fuente o binario	Codificación e Integración Continua (CI)	**Caja Blanca** (White Box)
SCA	Estática	Dependencias de terceros (librerías, frameworks)	Codificación e Integración Continua (CI)	**Caja Blanca** (Metadata)
IAST	Híbrida/Interactiva	Flujo de datos en tiempo de ejecución	Pruebas de QA y Funcionales	**Caja Gris** (Gray Box)
DAST	Dinámica	Aplicación en ejecución (interfaz de usuario)	Staging o Pre-producción	**Caja Negra** (Black Box)

Figura 3. Resumen de herramientas de análisis de seguridad (AST). Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

¿Por qué combinar estos tipos de análisis?

Cada técnica cubre **distintos espacios de riesgo** y tiene puntos fuertes y débiles. Combinándolas, se consigue:

- Obtener una lista de **requisitos de seguridad** que se van a implementar o que ya fueron implementados (según el enfoque si es durante SSDLC, que es lo ideal, o es una auditoría encargada después de terminar el SDLC de la aplicación).
- **Detección temprana** (SAST y SCA) para que los desarrolladores puedan corregir antes de que algo llegue a producción.
- Detección de errores que solo aparecen en ejecución real (DAST e IAST), como fallos en configuraciones, dependencias mal usadas, lógica del negocio, etc.
- **Menos falsos positivos** si se tiene contexto de ejecución (IAST) o si se validan ataques concretos (DAST).
- Mejora en la **visibilidad de la cadena** de *software* (SCA), lo cual es crítico ante vulnerabilidades en dependencias externas.
- Alineación con **requisitos de cumplimiento/auditorías,** pues muchos estándares exigen análisis de componentes, buen manejo del código, etc.
- Se pueden aprovechar sinergias entre herramientas para posibles **realimentaciones** de los **resultados** de tipo de análisis a otro posterior. Incluso algunos tipos de herramientas se puede ejecutar simultáneamente, como DAST e IAST, el *crawling* DAST lo puede aprovechar un agente IAST directamente.
- Se pueden usar **varias herramientas** del mismo tipo en **combinación,** ya que al tener distintos diseños pueden encontrar distintas vulnerabilidades y obtener un resultado más completo.

© Universidad Internacional de La Rioja (UNIR)

Metodología sugerida para un análisis combinado

En la Tabla 1 se muestra cómo se puede organizar y llevar a cabo un análisis de seguridad combinado:

Fase/actividad	Qué hacer	Herramientas/técnicas	Objetivo/resultado
Análisis/preparación	Definir el alcance: versiones, entornos (dev, *staging*, prod), tecnologías usadas, dependencias externas, API, servicios web, microservicios.	Inventario de tecnología, Lista de *software* (SBOM) preliminar, revisión del código, arquitectura.	Saber qué analizar y cómo.
Análisis diseño/revisión de los requisitos de seguridad	Se trata de investigar qué requisitos de seguridad se han propuesto para defender el aplicativo y cómo se pretende implementarlos.	Proyecto OWASP ASVS Modelado de amenazas.	Lista de requisitos de seguridad implementados y no implementados.
Desarrollo/SCA inicial	Identificar todas las dependencias (directas y transitorias), versiones, licencias. Comparar con bases de datos de vulnerabilidades.	SCA *tools*, generación de SBOM.	Identificar dependencias vulnerables y licencias problemáticas.
Desarrollo/SAST temprano *(shift-left)*	Ejecutar análisis estático sobre el código fuente/binaries. Integrarlo en el ciclo de desarrollo (IDE, *pre-commit*, *pull requests*).	Herramientas SAST.	Encontrar vulnerabilidades de codificación, patrones inseguros, errores comunes.
Pruebas/IAST (se puede realizar en conjunto con DAST)	Usar agentes o sensores durante pruebas funcionales/*integration testing*, y que el sistema esté ejecutándose con carga de pruebas reales o simuladas.	Herramientas IAST; pruebas automatizadas con cobertura buena.	Detectar vulnerabilidades que afectan al flujo de datos, lógicas de seguridad, rutas ejecutadas realmente.
Pruebas/DAST (se puede realizar en conjunto con IAST)	En un entorno seguro/explotable (*staging*, *staging-prod mirrored*, entornos de prueba), ejecutar análisis dinámico/simulaciones de ataque externos. Pruebas manuales de lógica de negocio si es necesario.	Herramientas DAST; *pentesting* complementario.	Detectar vulnerabilidades de interfaz, autenticación/autorización, inyección, XSS, CSRF, etc.

© Universidad Internacional de La Rioja (UNIR)

Test funcional de requisitos de seguridad	Basado en los requisitos definidos por el negocio o estándares, como OWASP ASVS *(application security verification standard)*.	Manual y automatizado.	Verificar que los controles de seguridad implementados cumplen con los requisitos definidos.
Correlación y priorización	Cruzar resultados de SAST, IAST, DAST, SCA. Ver cuáles vulnerabilidades son explotables, cuáles afectan código realmente ejecutado, cuáles vienen de dependencias externas. Priorizar según riesgo real, impacto, esfuerzo de corrección.	Herramientas que soporten *dashboard* unificado, correlación de hallazgos; uso de métricas, como cobertura de ejecución, gravedad, exposición.	Enfocar los esfuerzos donde más riesgo hay.
Remediación	Corregir *bugs* de código, actualizar dependencias, mejorar configuraciones; aplicar parches; escribir pruebas nuevas si faltaban.	Equipos de desarrollo, seguridad; integración en CI/CD.	Reducir la superficie de vulnerabilidad.
Repruebas/validación	Verificar que los *fixes* han funcionado; volver a ejecutar los análisis; asegurar que no se introducen regresiones.	*Re-run* de SAST, DAST, IAST; pruebas de integración.	Confirmación de que ya no existen los problemas.
Monitoreo continuo/mantenimiento	Revisar nuevas vulnerabilidades de dependencias, cobertura de pruebas, análisis de código nuevo; usar alertas; integración continua.	SCA continuo; escaneo automático; actualización de reglas; revisiones de código; auditorías periódicas.	Mantener la seguridad con el paso del tiempo.

Tabla 1. Metodología de análisis de seguridad (AST). Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

7.3. Software composite analysis (SCA)

Las herramientas de tipo *software composite analysis* (SCA) son soluciones especializadas que ayudan a **identificar, gestionar** y **asegurar** el uso de componentes de *software* de terceros, particularmente aquellos que son de código abierto *(open source)* dentro de una aplicación. Las herramientas SCA se centran en analizar el *composite software,* es decir, el conjunto de *software* que se construye a partir de **múltiples componentes,** usualmente bibliotecas y dependencias que no son desarrolladas internamente, sino integradas desde fuentes externas.

El objetivo principal es detectar **vulnerabilidades CVE,** problemas de **licencia** y **riesgos operacionales** asociados con esos componentes.

¿Qué analizan exactamente las herramientas SCA?

- **Dependencias:**
  - Bibliotecas de terceros utilizadas directa o indirectamente.
  - Incluye dependencias transitivas (aquellas que son requeridas por otras bibliotecas).
- **Licencias de *software:***
  - Identificación del tipo de licencia (MIT, GPL, Apache, etc.).
  - Evaluación de compatibilidad con las políticas legales de la empresa.
- **Vulnerabilidades conocidas CVE:**
  - Uso de bases de datos, como CVE, NVD, OSS Index, etc.
  - Mapeo de versiones de componentes a vulnerabilidades de seguridad.

© Universidad Internacional de La Rioja (UNIR)

- **Obsolescencia y mantenimiento:**
  - Componentes que ya no son mantenidos activamente.
  - Versiones desactualizadas.

¿Cómo funcionan?

Las herramientas SCA escanean el código fuente, archivos de proyecto y configuración para **detectar** (consultan bases de datos internas o públicas):

- **Archivos,** como package.json, pom.xml, requirements.txt, Gemfile, etc.
- **Binarios** y **contenedores** que puedan contener librerías incrustadas.
- **Composición** completa del *software* (incluso escaneo profundo, a veces llamado *deep dependency scanning*).
- **Comparan versiones de componentes** con vulnerabilidades conocidas.
- Verifican **compatibilidad de licencias.**
- Recomiendan **actualizaciones seguras** o cambios.

© Universidad Internacional de La Rioja (UNIR)

Ejemplos de herramientas SCA

Herramienta	Descripción breve
Snyk	Muy popular. Se integra fácilmente con CI/CD y repositorios. Fuerte enfoque en seguridad.
Black Duck (Synopsys)	Amplia cobertura de componentes y análisis de licencias. Muy usado en grandes empresas.
WhiteSource (ahora Mend)	Ofrece análisis en tiempo real y políticas personalizables.
Sonatype Nexus Lifecycle	Basado en la base de datos de Maven Central. Potente para proyectos en Java.
OWASP Dependency-Check	*Open source*. Detecta vulnerabilidades a partir de archivos de dependencias.
FOSSA	Foco en cumplimiento de licencias y seguridad. Se integra bien en *pipelines*.

Tabla 2. Ejemplo de herramientas SCA. Fuente: elaboración propia.

Beneficios del uso de SCA:

- **Seguridad proactiva:** identificación de vulnerabilidades antes de que lleguen a producción.
- **Cumplimiento legal:** control sobre el uso de licencias restrictivas (como GPL).
- **Mejor gestión de riesgos:** visibilidad sobre qué dependencias representan un riesgo y cómo mitigarlo.
- **Automatización en DevSecOps:** integración en *pipelines* CI/CD para alertar en tiempo real.
- **Mejor gobernanza del *software:*** alineación con políticas corporativas de seguridad y uso de código abierto.

© Universidad Internacional de La Rioja (UNIR)

Limitaciones:

- **Falsos positivos**: no todos los hallazgos son riesgos reales; se necesita revisión manual.
- **Cobertura variable**: algunas herramientas son más fuertes en ciertos lenguajes o ecosistemas.

© Universidad Internacional de La Rioja (UNIR)

7.4. Análisis de seguridad estático (SAST)

Para realizar el análisis de seguridad estático, se dispone de herramientas del tipo de **caja blanca** que analizan el **código fuente** y el ejecutable, según sea el caso: *static analysis security test* (SAST). Las que pueden analizar **código ejecutable** tienen primero que hacer un desensamblado para extraer el código fuente y, posteriormente, actúan como el otro tipo de herramientas estáticas de código fuente, es decir, tienen una etapa previa de conversión de código ejecutable en el código fuente correspondiente.

Por tanto, las consideraciones y el análisis de las de código fuente servirán también para las de código ejecutable y solo se comentarán para estas las particularidades y problemas que tiene la etapa previa de desensamblado para obtener el código fuente a partir del código ejecutable.

Las herramientas de tipo SAST toman como entrada el código fuente y lo trasforman partiendo de las **representaciones intermedias** (árbol de análisis sintáctico) resultantes de la fase compilación a un **modelo del código fuente** y, a continuación, lo analizan contra una serie de reglas definidas en las herramientas para generar los informes de error correspondientes. Este conjunto de reglas se puede aumentar en muchos casos por parte del usuario que puede definir las suyas propias para adaptarlas a las particularidades de la aplicación que se está analizando. Un ejemplo de lenguaje de especificación de reglas es PQL ***(program query language)*** para la definición de defectos de seguridad para lenguaje Java.

Este tipo de herramientas parten con un problema debido a que el hecho de determinar si un programa alcanza su estado final o no es un **problema indecidible** (Sipser, 2006). En este contexto, un falso positivo es un problema descubierto en un programa cuando en realidad no existe ninguno. La noción de utilizar un algoritmo para analizar otro es parte de los **orígenes de la computación.**

© Universidad Internacional de La Rioja (UNIR)

A pesar de este problema, tal y como dice McGraw (2006), estas herramientas son consideradas como una actividad muy importante de seguridad dentro de un SSDLC.

Características

Las herramientas de análisis estático **comprueban todo el código a fondo y coherentemente** sin ninguna tendencia, que a veces los programadores según su criterio podrían tener sobre algunas de las partes del código que pudieran ser más interesantes desde una perspectiva de seguridad o que pudieran ser más fáciles para realizar las pruebas dinámicas.

Un análisis valioso debe ser lo más imparcial posible. Examinar el código totalmente y a fondo supone una buena realimentación en el conocimiento de la aplicación ahondando más en el conocimiento de la seguridad de la aplicación.

Examinando el código en sí mismo, las herramientas de análisis estático pueden indicar la causa de origen de un problema de seguridad y no solamente uno de sus síntomas. Esto es importante para asegurarse de que las vulnerabilidades son corregidas correctamente.

El análisis estático **puede encontrar errores tempranamente en la fase de implementación y desarrollo** antes de que el programa sea ejecutado por primera vez. El encuentro de un error de este tipo no solo reduce el coste de arreglarlo, sino que además un ciclo de realimentación rápido puede ayudar a dirigir el trabajo de un programador. Este tiene la oportunidad de corregir errores de los que antes no era consciente. Los escenarios de ataque y la información sobre el código usados por una herramienta de análisis estático actúan como medio de transferencia de conocimiento.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Se ha comentado el problema que tienen en cuanto al reporte de **falsos positivos.** Esto se puede arreglar en gran medida mediante una auditoría posterior, para lo cual la **información de trace** que proporcionan las herramientas en muchos de los casos hace que esta tarea no sea tan complicada a la vez que aporta para la persona que la realiza una **realimentación de conocimiento** de la aplicación en su totalidad. La clasificación por grado de criticidad de las vulnerabilidades reportadas también supone una ayuda para esta auditoría posterior.

Los falsos positivos son seguramente indeseables, pero desde una perspectiva de seguridad, los **falsos negativos (defecto no descubierto)** son mucho peores. Con un falso negativo, un problema existe en el programa, pero la herramienta no lo detecta.

Según Stuttard (2008), la penitencia por un falso positivo es la cantidad de tiempo gastada repasando el resultado. La penitencia por un falso negativo es mucho mayor. No solo se paga el precio asociado por tener una vulnerabilidad en el código, se vive con un sentido falso de seguridad que se deriva del hecho de que la herramienta hizo parecer que todo era correcto. Todas las herramientas de análisis estático de código producen algunos falsos positivos o falsos negativos. La mayoría de ellas producen ambos. El balance que una herramienta efectúa entre falsos positivos y falsos negativos es normalmente indicativo del propósito de la herramienta.

Las herramientas SAST, por lo general, intentan producir un número bajo de falsos positivos y están más dispuestas a aceptar falsos negativos. La penitencia por las **vulnerabilidades de seguridad** pasadas por alto es elevada; por ello, otras herramientas SAST siguen el enfoque de reducir al mínimo los falsos negativos a costa de tener más falsos positivos que posteriormente se puedan descartar en la auditoría final de resultados.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

La existencia de falsos positivos y falsos negativos obliga a realizar una **auditoría posterior** de los informes de la herramienta que permitan eliminar los primeros y encontrar los segundos (bastante más complicado). Esto implica una adecuada formación en los defectos que se pueden cometer en el código de un determinado lenguaje de programación que puede ser más o menos amigable en función de las facilidades de **trace de la vulnerabilidad** que proporcione una determinada herramienta.

Ventajas

Detecta errores temprano, que es ideal para la integración en fases de desarrollo, cobertura del código completo y detectar patrones inseguros en lógica interna.

Inconvenientes

Puede generar muchos **falsos positivos,** por lo que necesita auditoría manual de los resultados; no ve lo que no se ejecuta; no detecta problemas de entorno o configuración en *runtime;* cobertura de pruebas/manuales necesarias para que SAST sea realmente eficaz.

Arquitectura

Las herramientas estáticas importan como entrada el código fuente y lo trasforman generando representaciones intermedias o modelos del código fuente según el caso y, a continuación, lo analizan contra una serie de reglas definidas en las herramientas, las cuales pueden realizar algunos o todos estos análisis:

- Análisis léxico, sintáctico y semántico como cualquier compilador.
- Análisis intraprocedural o local (dentro de cada función) del flujo de control y de los datos. Puede emplear técnicas o algoritmos como: métodos formales, interpretación abstracta (Fromherz *et al.* 2018), satisfacción *booleana* (Biere *et al.* 2018) o *model checking* (Beyer *et al.* 2018).

© Universidad Internacional de La Rioja (UNIR)

- Análisis global o interprocedural de llamadas entre funciones y flujo de los datos: *context-sensitive* (determinación del contexto de una función cuando es llamada), *path sensitive* (explora las rutas basado en información sobre el flujo de control), *path insensitive* (explora todas las rutas, es muy costoso) y basado en *function summaries* (utilización de resúmenes del contexto de llamada de las funciones. Es más flexible que la anterior, puede ser más o menos impreciso).

- El análisis global puede utilizar algoritmos como: *SAT solvers* (Biere *et al.* 2018), *theorem provers* (Khan *et al*. 2019) o *model checking* (Beyer *et al.* 2018).

El esquema de una **herramienta SAST genérica** se puede ver en la Figura 5 donde se evalúa la eficacia de nueve herramientas de análisis de seguridad del código fuente tanto comerciales como de *open source.* Estas fabrican un modelo del código que se analiza frente a las reglas de seguridad que caracterizan las vulnerabilidades para detectar apariciones de vulnerabilidades en el modelo y producir resultados.

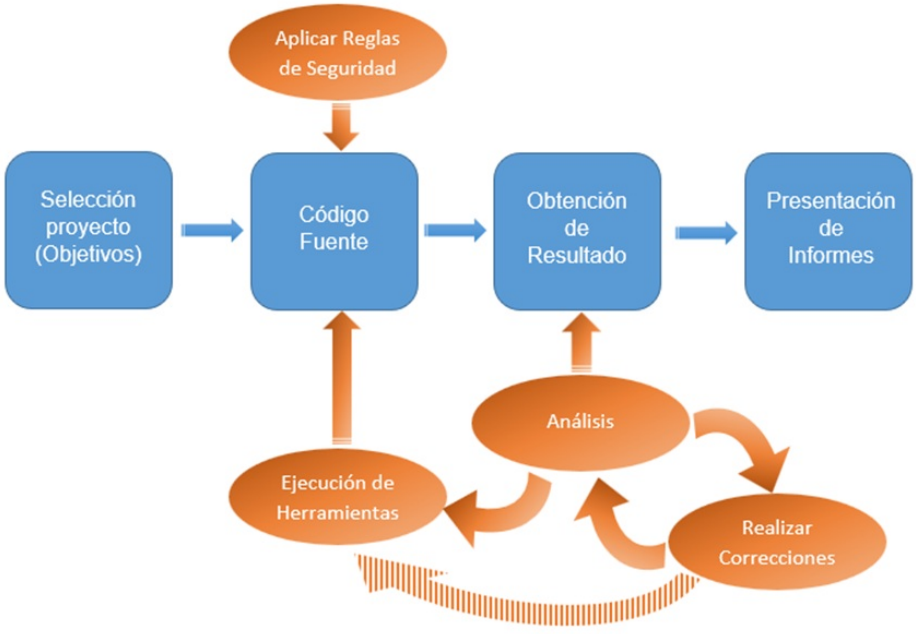


Figura 5. Arquitectura herramientas SAST. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Estas herramientas tienen un paso previo de **desensamblado del código ejecutable.** Una vez obtenido, analizan el código fuente conseguido de la misma forma que las comentadas anteriormente y, por tanto, se van a tratar principalmente las particularidades que tiene el desensamblaje del código ejecutable como paso previo al análisis. Estas aproximaciones analizan directamente el código máquina a partir de su simplificación para construir un **diagrama de control de flujo** y de llamadas.

Ejemplos SAST

Fortify SCA, blackduck SAST, HCL SAST, FindSecurityBugs, Xanitizer o Klocwork SonarQube Semgrep, Checkmarx SAST, Veracode SAST y PVS-Studio son buenos ejemplos de herramientas que suministran una muy buena información para, sobre todo, **eliminar falsos positivos.**

A continuación, puedes ver una lista de herramientas SAST: https://owasp.org/www-community/Source_Code_Analysis_Tools

---

© Universidad Internacional de La Rioja (UNIR)

7.5. Análisis funcional de requisitos de seguridad

Este tipo de análisis se concentra en realizar **pruebas** funcionales de **requisitos de seguridad.**

Objetivo

Verificar que los controles de seguridad implementados cumplen con los **requisitos definidos.**

- **Autenticación:**
  - *Multifactorauthentication.*
  - Políticas de contraseña.
- **Gestión de sesiones:** (*timeouts,* invalidación, etc.).
- **Autorización**:
  - Pruebas de control de acceso horizontal y vertical.
  - Roles y privilegios correctamente aplicados.
- **Gestión de errores:**
  - No divulgar detalles internos (*stack trace,* etc.).
  - Mensajes genéricos para usuarios finales.
- **Validación de entrada:**
  - Rechazo de datos maliciosos.
  - Escapes de caracteres.
  - Uso de listas blancas.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Protección de datos:**
  - Cifrado en tránsito y en reposo.
  - Seguridad de *cookies* (Secure, HttpOnly, Samesite).
- **Cabeceras de seguridad:**
  - HSTS: HTTP *strict transport security.*
  - CSP: *content security policy.*
  - X-*Frame-Options.*
  - X-*Content-Type.*
  - XSS-*Protection.*
  - CORS
- **Registro y monitoreo:**
  - Registro de eventos críticos.
  - Integración con SIEM.
  - Alertas de comportamiento anómalo y gestión de incidentes.
  - *Backup* y recuperación

Enfoque:

- Manual y automatizado usando cualquier tipo de herramienta de seguridad que pueda ayudar a comprobar que un **requisito de seguridad** está **bien implementado:** Proxy interceptador, Sniffer, Herramientas de fuerza bruta basadas en diccionario, *footprinting,* etc.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- Se basa en el análisis de la **completitud** de los **requisitos** definidos por el negocio realizado previamente en la fase de análisis del SSDLC usando proyectos, como: OWASP ASVS *(application security verification standard)*.

En el siguiente enlace, puedes acceder a la Guia para realizar una prueba funcional de seguridad OWASP WSTG: https://owasp.org/www-project-web-security-testing-guide/latest/6-Appendix/A-Testing_Tools_Resource

---

© Universidad Internacional de La Rioja (UNIR)

7.6. Análisis de seguridad dinámico (DAST)

En la fase de despliegue o de pruebas se debe llevar a cabo un reconocimiento de las capas de la aplicación usando la información de retroalimentación de las actividades de fases anteriores, como el modelado de amenazas, la derivación de requisitos de seguridad y la revisión de seguridad del código fuente.

Las herramientas de análisis dinámico de caja negra o gris (DAST) analizan una aplicación ejecutando la herramienta contra la aplicación en ejecución efectuando un *test* de penetración e intentando cubrir toda la superficie de ataque (todas las entradas posibles a la aplicación) para descubrir las vulnerabilidades que puedan existir. Como ejemplo de este tipo de herramientas para aplicaciones web se encuentran Webinspect, OWASP ZAP o Burp Suite.

Una herramienta DAST actúa como se muestra en la Figura 6. Configurada en modo **proxy**, se interpone entre el navegador y la aplicación web y lleva a cabo un descubrimiento del sitio web *(crawling)* para, posteriormente, lanzar ataques contra la aplicación, efectuando un *test* de penetración, inyectando datos y código maligno para detectar las vulnerabilidades.

© Universidad Internacional de La Rioja (UNIR)

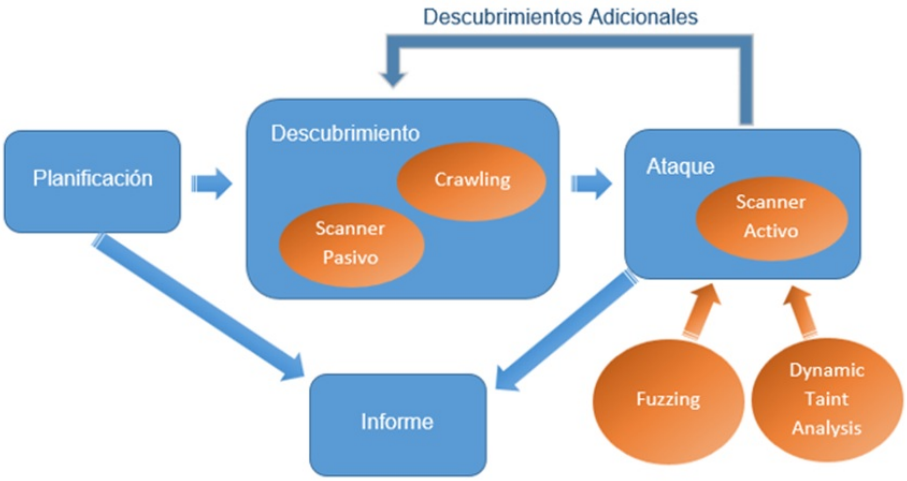


Figura 6. Dinámica de un escáner automático de vulnerabilidades. Fuente: adaptado de Souppaya y Scarfone, 2008.

Cubrir toda la superficie de ataque de la aplicación es difícil, el grado en el que esto se consigue determina también el de la eficacia de la herramienta. Es difícil porque hay que probar todas las entradas a la aplicación y con todos los roles de usuario, cada parámetro de cada solicitud y cada respuesta para encontrar el patrón de una vulnerabilidad.

Hay que entender las posibilidades y debilidades de cada herramienta DAST, conocer la herramienta para sacar el mejor partido posible interpretando sus resultados. Tienen sus limitaciones y pueden detectar un conjunto de vulnerabilidades debido a su naturaleza. Por ejemplo, son capaces de detectar importantes vulnerabilidades incluidas en el OWASP Top Ten Project, por ejemplo:

- XSS.
- SQLi.
- Path transversal.
- Command injection.

© Universidad Internacional de La Rioja (UNIR)

- Remote File Inclusión (RFI)
- Ausencia de cabeceras de seguridad
- Ausencia de token anti-CSRF
- Redirecciones no validadas
- Xpath injection.
- Http response spliting.

Debido a que realizan un análisis sintáctico de las peticiones y respuestas que se envían/reciben de la aplicación, no pueden comprender la semántica de varios parámetros en su conjunto que pueden esconder un intento de ataque. Por tanto, tienen dificultad en detectar otras, como vulnerabilidades de diseño relativas a la autenticación, autorización, etc.

Las herramientas de análisis dinámico DAST deberían reunir estas características:

- Ser capaces de identificar un subconjunto aceptable de vulnerabilidades de seguridad de las aplicaciones web.
- Soportar vectores de entrada HTML, XML, JSON.
- Soportar los métodos de autenticación más usados y capacidad proxy interceptador.
- Generar un informe para cada vulnerabilidad detectada, indicando una acción o conjunto de acciones que sugieren la citada vulnerabilidad.
- Tener ratios aceptables de verdaderos y falsos positivos.

© Universidad Internacional de La Rioja (UNIR)

Hay que tener presente el último punto relativo a los falsos positivos que estas herramientas pueden tener y que son necesarios de comprobar, al igual que ocurría con el análisis estático de código fuente. Una buena táctica puede ser correlar los resultados de análisis estáticos y los de los escáneres automáticos de aplicaciones web para ayudar en el descarte de falsos positivos. Otra aproximación es aprovechar los resultados del análisis estático para generar casos de test para el *scan* automático que permita comprobar la veracidad de la existencia de la vulnerabilidad reportada por el análisis estático.

En la página del sitio web **Web application scanners evaluation criteria** se dispone de un proyecto sobre criterios a tener en cuenta para la evaluación de estas herramientas y otros muchos recursos e información interesante. Asimismo, se puede consultar una amplia relación de las herramientas disponibles.

Una comparativa de herramientas de este tipo se puede consultar en el artículo «An empirical comparison of commercial and open-source web vulnerability scanners» de Amankwah (2020): https://onlinelibrary.wiley.com/doi/10.1002/spe.2870

En el trabajo se demuestra que la capacidad de detección de vulnerabilidades y el rendimiento varían determinando la capacidad de detección de vulnerabilidad de ocho WVS (tanto abiertas como comerciales) usando dos aplicaciones web vulnerables: WebGoat y la aplicación web Damn vulnerable (DVWA).

© Universidad Internacional de La Rioja (UNIR)

Las ocho herramientas DAST estudiadas fueron: Acunetix; HP WebInspect; IBM AppScan; OWASP ZAP; Skipfish, Arachni, Vega y IronWASP. El rendimiento se evaluó usando métricas de evaluación como: precisión; recall; índice de Youden; referencia web OWASP Web Benchamrk evaluación (WBE) y los criterios de evaluación DAST. Los resultados experimentales muestran que, mientras que los escáneres comerciales son eficaces en la detección de vulnerabilidades de seguridad, algunos de código abierto (como como ZAP y Skipfish) también puede ser eficaces.

Fases del análisis con una herramienta DAST

Las fases que llevar a cabo con una herramienta de análisis dinámico (DAST) son las siguientes:

- **Reconocimiento de las capas de la aplicación.** Consiste en averiguar todo lo posible con respecto a las capas de la aplicación, tecnologías usadas, OS, servidor de aplicaciones, SGBD, puertos que se utilizan, lenguajes de programación, de *scripting,* etc.

- ***Crawling.*** Es una fase de descubrimiento de la aplicación web, la estructura de directorios, formularios, etc. Mediante esta fase se obtiene su contenido accesible para poder atacarla posteriormente. Se debe hacer manualmente con la herramienta configurada en modo proxy y, a continuación, se realiza de forma automática añadiendo una configuración del contexto de análisis que incluye información de usuario administrador, método de autenticación utilizado, de gestión de sesiones en su caso, tecnologías, etc.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- ***Scan* pasivo**. Es un análisis de las cabeceras de las peticiones GET, POST a la aplicación web. Se examina la ausencia de determinados parámetros en las cabeceras que proporcionan seguridad como httponly, que prohíbe acceder a *cookies* si no es desde el protocolo HTTP directamente, es decir, prohíbe el acceso a *cookies* directamente desde *scripts.* Lo realiza automáticamente la herramienta a la vez que el *crawling* automático.

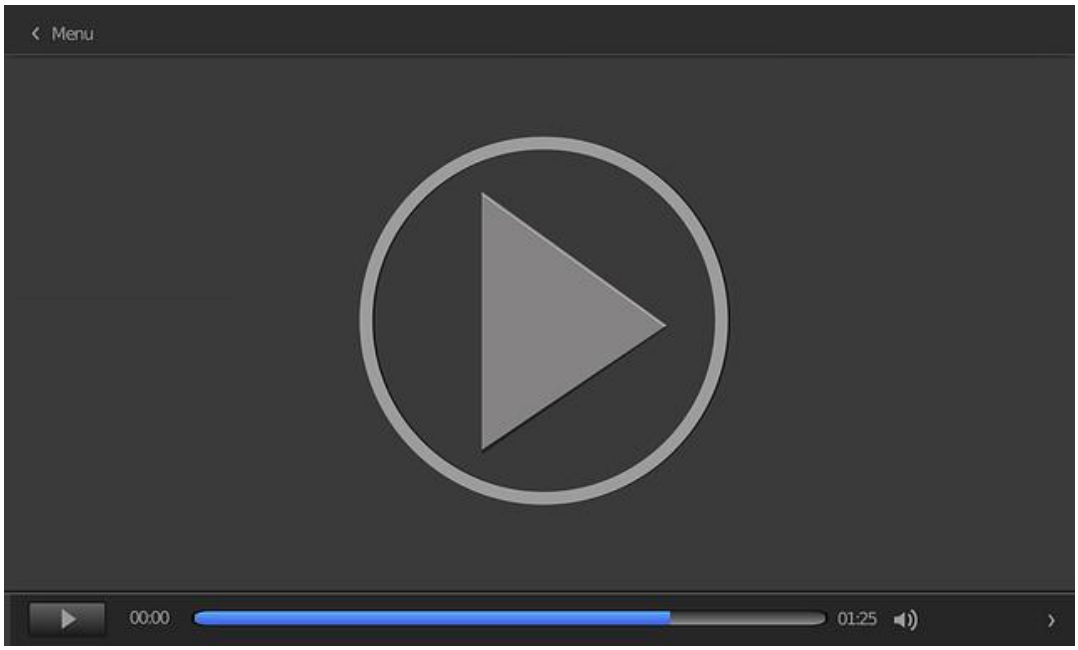
- ***Scan* activo.** Este *scan* inyecta *payloads* (*strings* maliciosos) en los campos de entrada accesibles de la aplicación y en base a un análisis sintáctico de las respuestas obtenidas en forma de posibles vulnerabilidades. En esta fase se aprovecha la configuración del contexto de la aplicación usada en la fase de *crawling* automático. Se necesita definir los objetivos inyectables (parámetros POST, GET, cabeceras, etc.), los posibles vectores de entrada (XML, JSON, etc.) y la política de *test.*

- **Auditoria de resultados.** Esta fase no es automática, la debe realizar manualmente el analista con ayuda de la herramienta. Las vulnerabilidades reportadas deben ser confirmadas por el analista o ingeniero de seguridad para descartar falsos positivos. El análisis que se hace de las respuestas de tipo sintáctico no es infalible. Se necesita repetir manualmente el ataque para confirmar cada vulnerabilidad con la ayuda de la herramienta. Finalmente, utilizando la herramienta, se debe analizar cada punto de la guía de *test* de OWASP (desde el punto 4.3 al punto 4.10).

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

En el vídeo *Herramientas DAST* hablaremos sobre cómo llevar a cabo una prueba de penetración a una aplicación web.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=7a3e9d80-c7d7-4077-a65c-adbf00bdbc6f

Lista de herramientas DAST: https://owasp.org/www-community/Vulnerability_Scanning_Tools

© Universidad Internacional de La Rioja (UNIR)

7.7. Análisis de seguridad interactivo (IAST)

Las herramientas de análisis interactivo (IAST) de caja blanca han surgido como un tipo de instrumento muy eficaz para descubrir vulnerabilidades en las aplicaciones web en combinación con los tipos de herramientas SAST y DAST, aunque con la diferencia de que el análisis del código fuente se realiza *runtime* y no de forma estática.

- **Las herramientas IAST se ejecutan en tiempo real** a través de un agente en el lado del servidor. Estas pueden tener acceso al código fuente, pueden instrumentar el código ejecutable, observar el entorno de ejecución de los procesos, analizar el contenido de sus variables en memoria y analizar el estado del proceso en general. Además, analizan las peticiones que se hacen a la aplicación web y las respuestas que se reciben. Estas acciones permiten detectar vulnerabilidades en los campos de entrada a la aplicación de forma concreta debido al estudio que, en tiempo real, se realiza sobre el funcionamiento de la aplicación. Se pueden aprovechar sinergias entre herramienta y las peticiones que envía el *crawling* DAST lo puede aprovechar un agente IAST directamente.



Figura 7. Arquitectura IAST. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Las **herramientas IAST** tienen parte de SAST y parte de DAST, pero solo conceptualmente, ya que el funcionamiento es diferente (a diferencia de SAST, el análisis del código fuente se realizar en rutntime). Una vez la herramienta detecta una vulnerabilidad, la aplicación realiza acciones en tiempo real, según Bermejo (2014):

- Se ejecuta en el servidor y obtiene los resultados del comportamiento que genera el usuario final sobre la aplicación publicada.
- Permiten la sanitización de las entradas, haciendo que la información que llega desde la parte cliente sea limpia, eliminando las posibles inyecciones o ejecuciones remotas.
- Emite alertas a los desarrolladores y administradores para que puedan corregirlas lo antes posible.
- La correlación de datos entre herramientas SAST e IAST suele ser más precisa al tratarse de herramientas de caja blanca.
- Generan un número menor de falsos positivos.

Un factor para tener en cuenta en este tipo de herramientas es que pueden **disminuir el rendimiento** de la aplicación al interferir su funcionamiento normal, sobre todo si la herramienta toma las acciones de bloquear o sanear, ya que estas acciones pueden sumar tiempo de cómputo a la aplicación que puede influir en el rendimiento disminuyendo el tiempo de respuesta.

© Universidad Internacional de La Rioja (UNIR)

Dentro de esta categoría se distinguen dos tipos de herramientas similares en concepto, pero con distinto enfoque de uso: **Interactive Analysis Security Test (IAST)** y **Real Analysis Self Protection (RASP).** Según el caso, una vez detectada la vulnerabilidad hay herramientas que pueden tomar una de las tres acciones siguientes:

- **Generar un informe (IAST),** después de la detección sin más. Securityscope de HP-Foritify es un ejemplo de este tipo. También Acutnetix+Acusensor, como funcionalidad añadida a ACUNETIX, que es un scanner automático de vulnerabilidades de aplicaciones web. Seeker, Contrast.

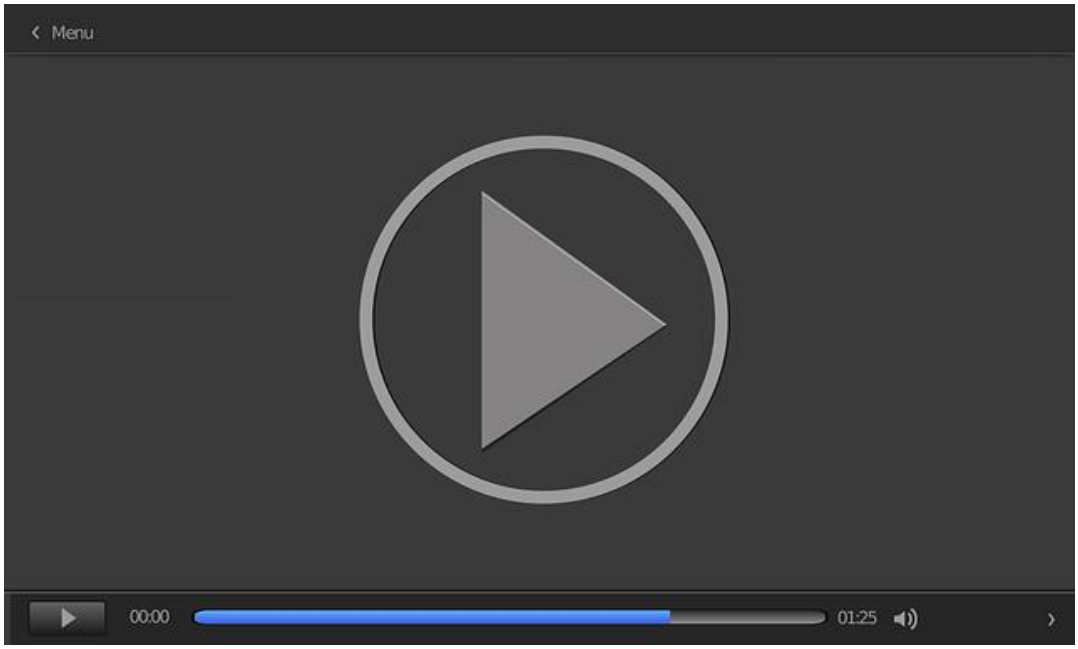
- **Bloquear (RASP)** los intentos de ataque, como por ejemplo Application Defender de HP-Fortify o Contrast RASP. Difieren en el concepto de lo que se hace cuando se detecta una vulnerabilidad: bloqueo o informar.

- **Sanear (RASP)** la petición maligna a la aplicación web, corrigiendo los valores de entrada a la aplicación. Saner es un ejemplo de prototipo de sanitización de entradas (Balzarotti *et al.* 2008).

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

En el vídeo *Herramienta IAST Contrast Comunity* presentaremos el tipo de herramientas de análisis de seguridad, que son dinámicas, denominadas Interactive Analysis Security Testing (IAST).



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=33f26579-634b-49da-9129-adcb00b6a4e1

© Universidad Internacional de La Rioja (UNIR)

Lista de herramientas IAST:

Herramienta	Descripción breve	Empresa
Contrast Security (Contrast Assess)	Una de las herramientas IAST más reconocidas. Se integra directamente en la aplicación para detectar vulnerabilidades en tiempo real mientras se ejecuta.	Contrast Security
Seeker	Proporciona análisis IAST enfocado en la trazabilidad de datos *(data flow analysis)* y cumplimiento de normativas, como PCI DSS y OWASP top 10.	Blackduck
HCL AppScan (con IAST)	Anteriormente IBM AppScan, ofrece capacidades IAST junto a SAST y DAST en un ecosistema integrado.	HCL Software
Veracode IAST	Parte de la suite Veracode, ofrece IAST como complemento a su análisis estático y dinámico. Se enfoca en entornos CI/CD.	Veracode
Fortify IAST (Runtime Application Self-Protection - RASP + IAST)	Fortify ofrece capacidades IAST como parte de su suite de seguridad. Ideal para grandes empresas.	OpenText (anteriormente Micro Focus)
K2 Cyber Security	Proporciona IAST con análisis basado en comportamiento en tiempo real y sin necesidad de escaneo tradicional.	K2 Cyber Security
Sqreen (ahora parte de Datadog)	Aunque originalmente más orientada a RASP, incluía capacidades IAST para detectar vulnerabilidades en tiempo de ejecución.	Datadog
Acunetix Premium (con integración IAST)	Aunque es una herramienta DAST, permite la integración con IAST a través de agentes, como AcuSensor.	Invicti
Imperva RASP/IAST	Ofrece IAST como parte de su plataforma de protección de aplicaciones, especialmente en entornos *cloud* y contenedores.	Imperva

Tabla 3. Lista de herramientas IAST. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

7.8. Herramientas de análisis de la seguridad híbridas (HAST)

Como resultado de la integración y realimentación de los tipos de herramientas anteriores, existen las llamadas **herramientas híbridas,** que son aquellas que presentan varias modalidades de funcionamiento, es decir, SAST + DAST, SAST + IAST o SAST+DAST+IAST. Permiten realizar dos tipos de escaneos. En ellas, se encuentra la combinación de **varias tecnologías en una sola,** haciendo que la herramienta sea más completa.

Las herramientas híbridas han ido evolucionando en el tiempo; a medida que lo hacían por separado SAST, DAST e IAST, permitiendo la correlación de resultados. Un ejemplo de este modo de funcionamiento sería la herramienta **Acunetix.** Esta permite realizar un análisis de código dinático (DAST) y posteriormente un análisis en tiempo real de la aplicación (IAST), pasando los resultados entre ellas para mejorar su rendimiento y aumentar el número de detecciones en el análisis de vulnerabilidades.

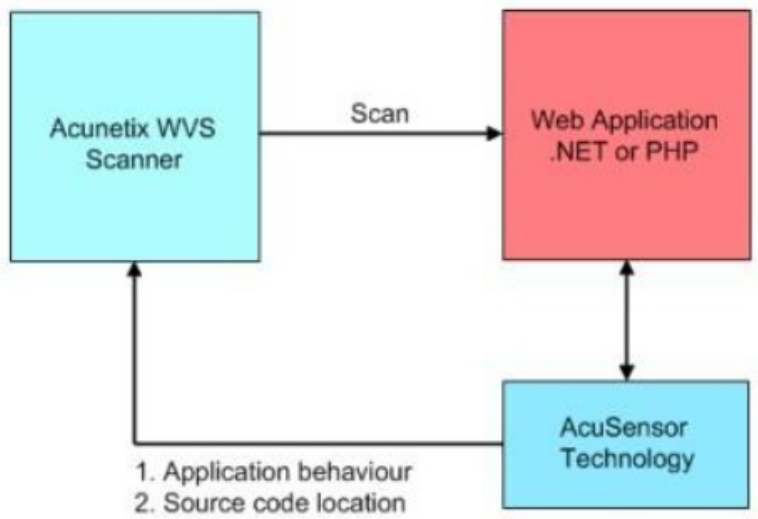


Figura 8. Arquitectura hibrida ACUNETIX+ACUSENSOR. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

Ejemplos de herramientas HAST

- Acunetix + Acusensor
- DerScanner
- Enlightn
- Fluid Attack's Scanner
- PT Application Inspector

© Universidad Internacional de La Rioja (UNIR)

7.9. Gestión y correlación de vulnerabilidades

Las herramientas de integración y correlación de vulnerabilidades de seguridad permiten **consolidar hallazgos** de vulnerabilidades provenientes de **múltiples fuentes** (SAST, DAST, IAST, SCA, *pentesting,* infraestructura, etc.). Pueden **correlacionar** esos hallazgos, eliminar duplicados y generar una vista unificada y priorizada. Automatizan la gestión del **ciclo de vida** de las vulnerabilidades en flujos DevSecOps o entornos empresariales, lo que facilita la comunicación entre seguridad, desarrollo y operaciones, y apoyan el cumplimiento normativo con trazabilidad, auditoría y métricas.

Integración con herramientas de escaneo:

- **SAST (análisis estático):** SonarQube, Fortify, Checkmarx, CodeQL.
- **DAST (análisis dinámico):** OWASP ZAP, Burp Suite, Netsparker.
- **SCA (análisis de componentes):** Snyk, WhiteSource, Dependency-Check.
- **IAST** (Contrast, Seeker).
- **Infraestructura/red:** Nessus, Qualys, OpenVAS.
- **Contenedores/*cloud:*** Trivy, Kube-hunter, Scout Suite.
- Estas integraciones pueden ser por API, importación de archivos, *plugins,* CI/CD, etc.

Correlación de hallazgos agrupando similares/dobles mediante:

- Identificadores (CVE, CWE, URI, etc.).
- *Hashes* de vulnerabilidad.

© Universidad Internacional de La Rioja (UNIR)

- Comparación de texto o patrones.
- Correlación multiherramienta (por ejemplo, misma vulnerabilidad detectada por Snyk y Trivy).

Priorización basada en riesgo:

- Uso de modelos, como CVSS, EPSS, VPR.
- Integración con fuentes externas (*threat intelligence,* contexto de negocio).
- Etiquetado de activos críticos vs. no críticos.

Gestión del ciclo de vida:

- **Estados:** activo, mitigado, falso positivo, aceptado.
- Asignación a **equipos de desarrollo.**
- Seguimiento de **acciones correctivas.**
- **Integración** con JIRA, GitHub Issues, GitLab, ServiceNow, etc.

Automatización DevSecOps:

- **Importación** automática en *pipelines* CI/CD.
- Umbrales de **calidad** de seguridad para permitir/desplegar código.
- **Alertas** y notificaciones (Slack, Teams, correo).
- API para orquestar **flujos personalizados.**

Reporting* y *compliance:

- *Dashboards* personalizables.
- Exportación de reportes (PDF, Excel, JSON).

© Universidad Internacional de La Rioja (UNIR)

- Indicadores clave (MTTR, vulnerabilidades abiertas por severidad, SLA).
- Evidencias para auditorías (ISO 27001, PCI-DSS, SOC 2, etc.).

Ejemplos de **herramientas:**

Herramienta	Tipo	Características destacadas	*Open source*
DefectDojo	Gestión de vulnerabilidades	Integración con múltiples AST, correlación automática, API *rest*, *reporting*	Sí
OWASP Glue	Gestión de vulnerabilidades	Integración con múltiples AST, correlación automática	Sí

Tabla 4. Lista de herramientas de correlación de vulnerabilidades. Fuente: elaboración propia.

© Universidad Internacional de La Rioja (UNIR)

7.10. Referencias bibliográficas

Amankwah, R., Chen, J., Kwaku, P. y Dave, K. (2020). An empirical comparison of commercial and open-source web vulnerability scanners. *Software Practice and Experience 50,* (9). https://doi.org/10.1002/spe.2870

Bermejo Higuera, J. R. (2014). *Metodología de evaluación de herramientas de análisis de seguridad de aplicaciones web para su adaptación en el ciclo de vida de desarrollo* [tesis doctoral, UNED]. Dialnet. https://dialnet.unirioja.es/servlet/tesis?codigo=44252

Beyer, D., Gulwani, S. y Schmidt, D. A. (2018). Combining model checking and data-flow analysis. En E. Clarke, T. Henzinger, H. Veith y R. Bloem (eds), *Handbook of model checking* (pp. 493-540). Springer. https://doi.org/10.1007/978-3-319-10575-8_16

Biere, A. y Kröning, D. (2018). SAT-Based Model Checking. En E. Clarke, T. Henzinger, H. Veith y R. Bloem (eds), *Handbook of model checking* (pp. 277-303). Springer. https://doi.org/10.1007/978-3-319-10575-8_10

Fromherz, A., Ouadjaout, A. y Miné, A. (2018, marzo 11). Static value analysis of Python programs by abstract interpretation. En A. Dutle, C. Muñoz Y A. Narkawicz (eds), *NASA formal methods* (pp. 185-202). Springer. https://doi.org/10.1007/978-3-319-77935-5_14

Khan, W., Kamran, M., Ahmad, A., Khan, F.A. y Derhab, A. (2019). A formal analysis of language-based Android security using theorem proving approach. *IEEE Access, 7,*
16550– 16560. https://www.researchgate.net/publication/330641040_Formal_Analysis_of_Language-Based_Android_Security_Using_Theorem_Proving_Approach

© Universidad Internacional de La Rioja (UNIR)

McGraw, G. (2006). *Software security: building security.* Addison Wesley Professional.

OWASP Web Security Testing Guide. (2020). https://github.com/OWASP/wstg

Sipser, M. (2006). *Introduction to the theory of computation* (2ª ed.). Thomson Course Technology.

Souppaya, M. P. y Scarfone, K. A. (2021). *Technical guide to information security testing and assessment.* National Institute of Standars and Technology. https://www.nist.gov/publications/technical-guide-information-security-testing-and-assessment

Stuttard, D. y Pinto, M. (2008). *The web application hacker's handbook: discovering and exploiting security flaws.* Wiley Publishing.

© Universidad Internacional de La Rioja (UNIR)

Análisis de seguridad estático en profundidad

Chess, B. y West, J. (2007). *Secure programming with static analysis.* Addison-Wesley.
https://livres.ycharbi.fr/Livres/ebook%20informatique/Informatique/Algorithmes%2C%20Programmation%20divers/Secure%20Programming%20with%20Static%20Analysis%20%282007%29%20-%20%5BAddison-Wesley%5D%20-%20Brian%20Chess%2C%20Jacob%20West.pdf

Este libro imprescindible de Brian Chess y Jacob West analiza los diferentes tipos de herramientas de análisis estático y cómo usarlas eficaz y eficientemente.

© Universidad Internacional de La Rioja (UNIR)

OWASP ZAP vídeos

All ZAP videos. (s. f.). https://www.zaproxy.org/videos-list/

Las herramientas de análisis dinámico permiten efectuar una prueba de penetración de una aplicación web al inyectar cadenas maliciosas en los campos de entrada de la aplicación. En este sitio web, se puede encontrar una comparativa de las principales herramientas de este tipo.

© Universidad Internacional de La Rioja (UNIR)

Herramientas de análisis de seguridad IAST

Página web de Contrast Security (https://www.contrastsecurity.com/contrast-community-edition).

Las herramientas de análisis interactivo realizan una prueba de seguridad de la aplicación en tiempo real embebidas en el servidor de aplicaciones. Contrast Comunity Edition es una versión demo que puede servir para comprobar la eficacia de este tipo de herramientas.

© Universidad Internacional de La Rioja (UNIR)

Comparativa de eficacia de herramientas SAST, DAST e IAST

Tudela, F. M., Bermejo Higuera, J. R., Bermejo Higuera, J., Sicilia Montalvo, J. A. y Argyros, M. I. (2020). On combining static, dynamic and interactive analysis security testing tools to improve OWASP top ten security vulnerability detection in web applications. *Applied Sciences, 10*(24), 9119. https://www.mdpi.com/2076-3417/10/24/9119

El diseño de las técnicas y algoritmos utilizados por las herramientas de pruebas de seguridad estáticas, dinámicas e interactivas difiere. Por lo tanto, cada herramienta detecta, en mayor o menor medida, cada tipo de vulnerabilidad para la que ha sido diseñada. Además, sus diferentes diseños hacen que tengan diferentes porcentajes de falsos positivos. Con el fin de aprovechar las posibles sinergias que pueden tener los diferentes tipos de herramientas de análisis, este artículo combina varias herramientas de pruebas de seguridad de análisis estático, dinámico e interactivo: análisis de seguridad estático de caja blanca (SAST), análisis de seguridad dinámico de caja negra (DAST) y análisis de seguridad interactivo de caja blanca (IAST), respectivamente. El objetivo es investigar cómo mejorar la eficacia de la detección de vulnerabilidades de seguridad y reducir el número de falsos positivos. En concreto, se combinarán dos herramientas de análisis de seguridad estática, dos dinámicas y dos interactivas para estudiar su comportamiento utilizando un punto de referencia específico para las diez vulnerabilidades de seguridad más importantes de OWASP y teniendo en cuenta diversos escenarios de diferente criticidad en términos de las aplicaciones analizadas. Por último, este estudio analiza y debate los valores de las métricas seleccionadas aplicadas a los resultados para cada combinación de herramientas.

© Universidad Internacional de La Rioja (UNIR)

Aplicaciones web para aprendizaje. Juice Shop

OWASP Juice Shop. (s. f.). https://owasp.org/www-project-juice-shop/

Juice Shop es una aplicación deliberadamente insegura que permite a los desarrolladores interesados, como usted, probar las vulnerabilidades que se encuentran comúnmente en las aplicaciones basadas en NodeJs, Angular, Mongodb, que utilizan componentes comunes y populares de código abierto.

© Universidad Internacional de La Rioja (UNIR)

Web Application Security Testing Guide

OWASP Web Security Testing Guide. (s. f.). https://owasp.org/www-project-web-security-testing-guide/

Guía de seguridad de OWASP.

© Universidad Internacional de La Rioja (UNIR)

Test

1. ¿Qué tipos de análisis puede realizar una herramienta SAST para comprobar las interacciones entre distintas funciones?

A. Intraprocedural.

B. Interprocedural.

C. Semántico.

D. Todos los anteriores.

2. ¿Qué tipo de análisis realiza una herramienta de tipo DAST?

A. Semántico.

B. Sintáctico.

C. Interprocedural

D. Intraprodedural.

3. Las herramientas de tipo IAST pueden instrumentar el:

A. Código fuente.

B. Peticiones y respuestas.

C. Código ejecutable.

D. Ninguna de las anteriores.

4. La prueba funcional de seguridad se realiza en fase de:

A. Análisis.

B. Diseño.

C. Desarrollo.

D. Pruebas o despliegue.

© Universidad Internacional de La Rioja (UNIR)

5. El análisis estático de código fuente se realiza en la fase de:

   A. Análisis.

   B. Diseño.

   C. Desarrollo.

   D. Pruebas o despliegue.

6. El análisis estático de caja blanca:

   A. Realiza un análisis local de cada función.

   B. Realiza un análisis interprocedural entre clases o módulos.

   C. Realiza un análisis léxico, sintáctico y semántico del código fuente y configuraciones.

   D. Todas las anteriores.

7. Señalar la afirmación falsa:

   A. Las herramientas de análisis de tipo IAST no suelen tener falsos positivos.

   B. Las herramientas RASP se pueden utilizar como *firewalls* de aplicaciones web de tipo *software.*

   C. Las herramientas de tipo IAST normalmente no tienen impacto en el rendimiento.

   D. Las herramientas IAST obtienen un informe del análisis de vulnerabilidades.

8. Las herramientas de tipo RASP:

   A. Bloquean las peticiones maliciosas que son capaces de explotar una vulnerabilidad.

   B. No bloquean, solo detectan vulnerabilidades.

   C. Tienen muchos falsos positivos.

   D. Todas las anteriores son ciertas.

© Universidad Internacional de La Rioja (UNIR)

9. Las herramientas de tipo RASP se usan en la fase de:

A. Análisis.

B. Diseño.

C. Desarrollo.

D. Producción.

10. Señalar la afirmación correcta en cuanto a herramientas de análisis de la seguridad SAST.

A. Las herramientas SAST no cubren todo en código de la aplicación.

B. Las herramientas SAST tienen falsos negativos y falsos positivos.

C. Las herramientas SAST no pueden analizar código ejecutable.

D. Todas las anteriores son falsas.

© Universidad Internacional de La Rioja (UNIR)